

Metody programowania

mgr inż. Katarzyna Dadek

2025

Laboratorium 8: Wzorce projektowe cz. 2

Na poprzednim laboratorium skupiliśmy się na **wzorcach kreacyjnych**, które pomagają kontrolować proces tworzenia obiektów. W tej części przechodzimy do **wzorców strukturalnych** i **behawioralnych**, które porządkują sposób, w jaki klasy i obiekty **łączą się** oraz **współpracują** ze sobą w czasie działania programu.

8.1 Wzorce strukturalne

Notatka

Wzorce strukturalne opisują sposób organizacji klas i obiektów w większe, logiczne struktury. Skupiają się na tym, jak coś jest zbudowane i jak elementy systemu można ze sobą łączyć, aby:

- uprościć zależności,
- ukryć złożoność,
- umożliwić łatwe modyfikacje i rozszerzenia.

8.1.1 Przykładowe wzorce strukturalne

- **Adapter** – dostosowuje jedno API do drugiego.
- **Dekorator** – dodaje nowe funkcjonalności obiektom bez zmiany ich klasy.
- **Kompozyt** – reprezentuje struktury drzewiaste (część-całość).
- **Fasada** – upraszcza dostęp do złożonego systemu.
- **Pełnomocnik (Proxy)** – pośrednik kontrolujący dostęp do obiektu.
- **Most (Bridge)** – oddziela abstrakcję od implementacji.

Wskazówka

Wzorce strukturalne są szczególnie przydatne, gdy:

- system staje się **przepełniony zależnościami**,
- komunikacja między modułami jest **mocno sprzężona**,
- chcesz **opakować** istniejący kod bez jego modyfikacji.

8.1.2 Adapter

Adapter to wzorec, który **pozwala współpracować klasom o niezgodnych interfejsach**. Działa jak **tłumacz** – zamienia jedno API na inne, tak aby klient mógł używać obiektu, nie wiedząc, że wewnątrz dzieje się „konwersja”.

Przykładowo: Twoja aplikacja oczekuje obiektu z metodą `playAudio(file)`, ale masz bibliotekę, która udostępnia tylko `play(filePath: string)`. Zamiast przerabiać kod klienta lub biblioteki, tworzysz **adapter**, który implementuje oczekiwany interfejs i wewnętrznie deleguje wywołania do odpowiedniej biblioteki.

Notatka

Adapter:

- Łączy ze sobą **niekompatybilne interfejsy**.
- Umożliwia **ponowne użycie** istniejących klas bez modyfikacji.
- Izoluje kod klienta od szczegółów implementacji.

Wskazówka

Przykładowa implementacja wzorca **Adapter** przedstawiona jest w dodatku A.

8.1.3 Dekorator

Dekorator pozwala **dynamicznie dodawać nowe odpowiedzialności** obiektom bez zmieniania ich klasy. Zamiast tworzyć dziesiątki klas potomnych z różnymi kombinacjami zachowań, możesz **opakowywać** obiekt dodatkowymi dekoratorami. Przykład: system logowania:

- dekorator dodający zapis do pliku,
- dekorator dodający wysyłkę do zewnętrznego systemu,
- dekorator formatujący wiadomości itp.

Notatka

Dekorator:

- Utrzymuje **ten sam interfejs** co dekorowany obiekt.
- Otacza obiekt „warstwą” dodatkowego zachowania.
- Umożliwia **kompozycję funkcjonalności** zamiast dziedziczenia.

Wskazówka

Przykładowa implementacja wzorca *Dekorator* przedstawiona jest w dodatku B.

8.1.4 Kompozyt

Kompozyt służy do reprezentowania **struktur drzewiastych**, w których obiekty mogą być:

- **liśćmi** – pojedynczymi elementami,
- **węzłami** – złożonymi elementami zawierającymi inne obiekty.

Kluczowe jest to, że **klient operuje na liściach i węzłach w ten sam sposób**, ponieważ implementują one wspólny interfejs. Typowy przykład to system plików (foldery i pliki) albo drzewo UI (komponenty i kontenery).

Notatka

Kompozyt:

- Ujednolica obsługę **pojedynczych i złożonych** obiektów.
- Pozwala łatwo tworzyć i przetwarzać **struktury drzewiaste**.
- Uproszcza logikę klienta – brak potrzeby rozróżniania typów.

Wskazówka

Przykładowa implementacja wzorca *Kompozyt* przedstawiona jest w dodatku C.

8.1.5 Fasada

Fasada zapewnia **uproszczony interfejs** do złożonego systemu klas. Zamiast zmuszać klienta do korzystania bezpośrednio z wielu modułów (np. kilkunastu serwisów, klas pomocniczych, konfiguracji), dostarczasz **jedną klasę-fasadę**, która „załatwia wszystko”.

Notatka

Fasada:

- Ukrywa **złożoność** systemu za prostym interfejsem.
- Zmniejsza liczbę zależności po stronie klienta.
- Ułatwia **modułowość i refaktoryzację**.

8.1.6 Pełnomocnik (Proxy)

Pełnomocnik to obiekt, który **kontroluje dostęp** do innego obiektu. Może:

- opóźniać tworzenie obiektu (leniwa inicjalizacja),
- cachować wyniki,
- sprawdzać uprawnienia,
- komunikować się z obiektem zdalnym (np. przez sieć).

8.2 Wzorce behawioralne

Notatka

Wzorce behawioralne opisują **sposób komunikacji i podział odpowiedzialności** między obiektami. Skupiają się na tym, co obiekty robią i jak ze sobą współpracują, aby:

- uprościć przepływ danych i zdarzeń,
- zredukować sprzężenie,
- ułatwić modyfikacje zachowań bez przepisywania całego systemu.

8.2.1 Przykładowe wzorce behawioralne

- **Strategia** – wymienne algorytmy.
- **Obserwator** – powiadamianie wielu obiektów o zmianie stanu.
- **Polecenie (Command)** – zamiana operacji w obiekty.
- **Stan** – zmiana zachowania obiektu w zależności od jego stanu.
- **Łańcuch zobowiązań** – przekazywanie żądania po łańcuchu handlerów.
- **Metoda szablonowa (Template Method)** – definicja szkieletu algorytmu z możliwością nadpisania kroków.

8.2.2 Strategia

Strategia pozwala **wydzielić algorytm** do osobnej klasy i sprawić, że będzie on **wymienny w czasie działania programu**. Obiekt korzystający z algorytmu (kontekst) nie musi wiedzieć, jak dokładnie jest on zaimplementowany – wystarczy, że zna interfejs strategii.

Przykłady:

- różne sposoby sortowania,
- różne sposoby obliczania ceny (promocje, rabaty),
- różne algorytmy kompresji lub szyfrowania.

Notatka

Strategia:

- Umożliwia łatwą wymianę algorytmów.
- Zapewnia **wspólny interfejs** dla różnych implementacji.
- Zmniejsza liczbę instrukcji warunkowych typu `if/else/switch`.

Wskazówka

Przykładowa implementacja wzorca *Strategia* przedstawiona jest w dodatku D.

8.2.3 Obserwator

Obserwator opisuje relację **jeden–wiele** między obiektami. Gdy obiekt–**obserwowany** zmienia swój stan, powiadamia automatycznie wszystkie **obserwatory**, które mogą zareagować (np. zaktualizować widok). Typowe zastosowania:

- GUI (model powiadamia widoki),
- systemy eventowe,
- notyfikacje (np. subskrypcje).

Notatka

Obserwator:

- Rozdziela **źródło danych** od ich odbiorców.
- Umożliwia dynamiczne **dodawanie/usuwanie obserwatorów**.
- Ułatwia implementację systemów reaktywnych.

Wskazówka

Przykładowa implementacja wzorca *Obserwator* przedstawiona jest w dodatku E.

8.2.4 Polecenie (Command)

Wzorzec Polecenie zamienia **żądania/operacje w osobne obiekty**. Dzięki temu można:

- kolejkować polecenia,
- cofać operacje (undo),
- logować historię,
- przekazywać polecenia między warstwami systemu.

Przykład: w aplikacji GUI przyciski mogą wykonywać różne polecenia, a każde polecenie wie, jak wykonać i ewentualnie odwrócić swoją operację.

Notatka

Polecenie:

- Hermetyzuje wywołanie operacji w obiekcie.
- Ułatwia implementację undo/redo.
- Umożliwia kolejkowanie i logowanie zleceń.

Wskazówka

Przykładowa implementacja wzorca *Polecenie* przedstawiona jest w dodatku F.

8.2.5 Stan

Stan pozwala obiektowi **zmieniać swoje zachowanie w zależności od aktualnego stanu**, bez rozbudowanych instrukcji warunkowych. Zamiast pisać:

```
if (state === "Draft") { ... } else if (state === "Published") { ... }
```

tworzysz osobne klasy reprezentujące stany i przenosisz do nich logikę.

8.2.6 Łańcuch zobowiązań

Łańcuch zobowiązań polega na przekazywaniu żądania przez **łańcuch obiektów**, z których każdy może **obsłużyć żądanie** i/lub **przekazać je dalej**. Przykład: filtracja requestów HTTP, walidacja, obsługa błędów.

8.3 Sprawozdanie 8

W oparciu o dotychczas stworzoną aplikację, **zidentyfikuj miejsca, w których sposób współpracy obiektów lub struktura klas zaczyna być problematyczna** (nadmiar zależności, rozbudowane instrukcje warunkowe, powtarzalny kod, trudności z rozbudową).

- Wybierz **co najmniej jeden** fragment kodu, w którym:
 - obiekty są **mocno sprzężone**,
 - wykonuje się **wiele instrukcji if/else/switch** zależnych od typu lub stanu,
 - architektura utrudnia **testowanie** i **rozszerzanie** zachowań.
- Zastosuj **odpowiedni wzorzec strukturalny lub behawioralny** (np. Adapter, Dekorator, Kompozyt, Strategia, Obserwator, Polecenie), aby:
 - zmniejszyć sprzężenie między klasami,
 - uprościć przepływ logiki,
 - ułatwić dodawanie nowych wariantów zachowań.
- Porównaj **stan przed i po refaktoryzacji**:
 - co było problemem,
 - jaki wzorzec został użyty,
 - jakie korzyści dało jego zastosowanie (czytelność, testowalność, możliwość rozbudowy).

Na koniec przygotuj **krótkie podsumowanie**, w którym opiszesz:

- jakie wzorce zostały użyte,
- jakie problemy projektowe udało się rozwiązać,
- jak te zmiany wpływają na dalszy rozwój i utrzymanie systemu.

A Adapter

```
1  interface AudioPlayer {
2      playAudio(file: string): void;
3  }
4
5  class LegacyPlayer {
6      play(filePath: string): void {
7          console.log("Playing from path:", filePath);
8      }
9  }
10
11 // Adapter
12 class LegacyPlayerAdapter implements AudioPlayer {
13     private legacy: LegacyPlayer;
14
15     constructor(legacy: LegacyPlayer) {
16         this.legacy = legacy;
17     }
18
19     playAudio(file: string): void {
20         this.legacy.play(file);
21     }
22 }
23
24 const legacy = new LegacyPlayer();
25 const player: AudioPlayer = new LegacyPlayerAdapter(legacy);
26 player.playAudio("music.mp3");
```


B Dekorator

```
1 interface Notifier {
2     send(message: string): void;
3 }
4
5 class BasicNotifier implements Notifier {
6     send(message: string): void {
7         console.log("Base notification:", message);
8     }
9 }
10
11 // Dekorator bazowy
12 abstract class NotifierDecorator implements Notifier {
13     protected wrappee: Notifier;
14
15     constructor(wrappee: Notifier) {
16         this.wrappee = wrappee;
17     }
18
19     send(message: string): void {
20         this.wrappee.send(message);
21     }
22 }
23
24 class EmailNotifier extends NotifierDecorator {
25     send(message: string): void {
26         super.send(message);
27         console.log("Sending email:", message);
28     }
29 }
30
31 class SmsNotifier extends NotifierDecorator {
32     send(message: string): void {
33         super.send(message);
34         console.log("Sending SMS:", message);
35     }
36 }
37
38 let notifier: Notifier = new BasicNotifier();
39 notifier = new EmailNotifier(notifier);
40 notifier = new SmsNotifier(notifier);
41
42 notifier.send("System alert!");
```

C Kompozyt

```
1 interface FileSystemItem {
2     getName(): string;
3     print(indent?: string): void;
4 }
5
6 class File implements FileSystemItem {
7     private name: string;
8
9     constructor(name: string) {
10         this.name = name;
11     }
12
13     getName(): string {
14         return this.name;
15     }
16     print(indent: string = ""): void {
17         console.log(indent + "- " + this.name);
18     }
19 }
20
21 class Folder implements FileSystemItem {
22     private name: string;
23     private children: FileSystemItem[] = [];
24
25     constructor(name: string) {
26         this.name = name;
27     }
28
29     add(item: FileSystemItem): void {
30         this.children.push(item);
31     }
32     getName(): string {
33         return this.name;
34     }
35     print(indent: string = ""): void {
36         console.log(indent + "+ " + this.name);
37         for (const child of this.children) {
38             child.print(indent + " ");
39         }
40     }
41 }
42
```

```
43  const root = new Folder("root");
44  const src = new Folder("src");
45  const file1 = new File("index.ts");
46  const file2 = new File("app.ts");
47
48  src.add(file1);
49  src.add(file2);
50  root.add(src);
51  root.print();
```

D Strategia

```
1  interface PriceStrategy {
2      calculate(basePrice: number): number;
3  }
4
5  class RegularPrice implements PriceStrategy {
6      calculate(basePrice: number): number {
7          return basePrice;
8      }
9  }
10
11 class BlackFridayPrice implements PriceStrategy {
12     calculate(basePrice: number): number {
13         return basePrice * 0.5;
14     }
15 }
16
17 class StudentDiscountPrice implements PriceStrategy {
18     calculate(basePrice: number): number {
19         return basePrice * 0.8;
20     }
21 }
22
23 class Cart {
24     private strategy: PriceStrategy;
25
26     constructor(strategy: PriceStrategy) {
27         this.strategy = strategy;
28     }
29
30     setStrategy(strategy: PriceStrategy): void {
31         this.strategy = strategy;
32     }
33
34     getTotal(basePrice: number): number {
35         return this.strategy.calculate(basePrice);
36     }
37 }
38
39 const cart = new Cart(new RegularPrice());
40 console.log(cart.getTotal(100)); // 100
41
42 cart.setStrategy(new BlackFridayPrice());
```

```
43 console.log(cart.getTotal(100)); // 50
44
45 cart.setStrategy(new StudentDiscountPrice());
46 console.log(cart.getTotal(100)); // 80
```

E Observer

```
1 interface Observer {
2     update(message: string): void;
3 }
4
5 interface Subject {
6     subscribe(observer: Observer): void;
7     unsubscribe(observer: Observer): void;
8     notify(message: string): void;
9 }
10
11 class NewsChannel implements Subject {
12     private observers: Observer[] = [];
13
14     subscribe(observer: Observer): void {
15         this.observers.push(observer);
16     }
17     unsubscribe(observer: Observer): void {
18         this.observers = this.observers.filter(o => o !== observer);
19     }
20     notify(message: string): void {
21         for (const observer of this.observers) {
22             observer.update(message);
23         }
24     }
25     publishNews(message: string): void {
26         console.log("News:", message);
27         this.notify(message);
28     }
29 }
30
31 class User implements Observer {
32     private name: string;
33
34     constructor(name: string) {
35         this.name = name;
36     }
37
38     update(message: string): void {
39         console.log(`${this.name} received: ${message}`);
40     }
41 }
42
```

```
43  const channel = new NewsChannel();
44  const alice = new User("Ala");
45  const bob = new User("Bartek");
46
47  channel.subscribe(alice);
48  channel.subscribe(bob);
49
50  channel.publishNews("New article!");
```

F Polecenie (Command)

```
1 interface Command {
2     execute(): void;
3     undo(): void;
4 }
5
6 class Light {
7     on(): void {
8         console.log("Light is ON");
9     }
10
11     off(): void {
12         console.log("Light is OFF");
13     }
14 }
15
16 class LightOnCommand implements Command {
17     private light: Light;
18
19     constructor(light: Light) {
20         this.light = light;
21     }
22
23     execute(): void {
24         this.light.on();
25     }
26
27     undo(): void {
28         this.light.off();
29     }
30 }
31
32 class LightOffCommand implements Command {
33     private light: Light;
34
35     constructor(light: Light) {
36         this.light = light;
37     }
38
39     execute(): void {
40         this.light.off();
41     }
42 }
```



```
43     undo(): void {
44         this.light.on();
45     }
46 }
47
48 class Remote {
49     private history: Command[] = [];
50
51     run(command: Command): void {
52         command.execute();
53         this.history.push(command);
54     }
55
56     undoLast(): void {
57         const command = this.history.pop();
58         if (command) {
59             command.undo();
60         }
61     }
62 }
63
64 const light = new Light();
65 const remote = new Remote();
66
67 const on = new LightOnCommand(light);
68 const off = new LightOffCommand(light);
69
70 remote.run(on); // Light is ON
71 remote.run(off); // Light is OFF
72
73 remote.undoLast(); // Light is ON
```